ISPD2027 Contest: MCMM Post-CTS Buffering and Sizing

Lead: Dr. Yiting Liu, UCSD ABKGroup [yil375@ucsd.edu]

Table of Contents

1. Contest Introduction
2. Contest Objectives
3. Problem Formulation
   - 3.1. Input Formats
   - 3.2. Output Formats
   - 3.3. Evaluation
4. Submission Guidelines
5. Benchmarks
6. Timeline
7. Contact
8. Registration
9. Contest Prizes
10. Organizers
11. Sponsors

References

==========

1. Contest Introduction

Repeater insertion (buffers and inverters) and gate sizing are two important approaches for timing optimization in VLSI physical design. The ISPD26 Contest [1] challenged contestants to develop **post-placement buffering and sizing** methods under practical physical constraints, including fixed macros and I/Os, PDN and placement blockages, limited routing resources, placement legality, and ERC requirements.

The ISPD27 Contest builds on ISPD26 and scales up the realism of the timing optimization problem. The central extension is to move the contestant optimization point to the **post-clock-tree-synthesis (post-CTS) stage** and evaluate solutions under **multi-corner multi-mode (MCMM)** timing constraints with propagated clocks. Contestants must therefore make buffering and sizing decisions that improve setup timing while remaining robust across multiple operating scenarios and avoiding unacceptable hold-timing degradation.

The major extensions from ISPD26 are:

- Post-CTS optimization
  - Contestants optimize a legalized post-CTS design.

    - Timing analysis uses propagated clocks instead of ideal clocks.
- Multi-Corner Multi-Mode (MCMM) timing
    - Buffering and sizing decisions must perform well across multiple timing scenarios.
- Setup and hold timing
    - Setup timing remains the primary optimization target.
    - Hold timing is considered as a constraint/penalty to prevent setup improvement at the cost of severe hold violations.
- Additional logic-preserving optimization
    - Equivalent input-pin swapping will be supported.
- Stronger reference baseline
    - The 2027 contest will use an updated OpenROAD-based reference flow with stricter legality checks, improved ERC validation, and more robust physical-design configurations.
- More robust submission and evaluation
    - Final evaluation will use separate hidden testcases in a controlled evaluation environment.
    - Submission requirements and validation checks will be strengthened to improve reproducibility, reduce evaluation turnaround time, and ensure fairness.

This contest is built on top of the OpenROAD [2] infrastructure and provides a controlled, open-source environment for benchmarking and evaluation. To encourage scalability and innovation, contestants may (but are not required to) leverage **GPU-accelerated** solutions or **modern ML** infrastructure such as PyTorch [3], which have shown strong potential in solving large scale optimization problems in physical design [4]-[11]. Nonetheless, the primary objective of this contest remains achieving **better PPA results under various constraints.**

2. Contest Objectives

- The contest focuses on post-CTS optimization for better PPA and timing robustness via:
    - Repeater optimization
        - Insert, remove, or replace data-path buffers and inverters to improve timing and reduce electrical-rule violations.
    - Gate sizing
        - Select appropriate drive-strength and VT variants from the provided cell libraries.
    - Equivalent-pin swapping
        - Swap logically equivalent cell input pins for timing improvement.
- Constraints
    - Inverter removal or replacement must preserve signal polarity.
    - Macros and I/Os are fixed.
    - PDN and placement blockages must be honored.
    - Routing track supply is fixed.

    - The provided clock topology, clock-cell masters and clock-cell locations are fixed. Contestants cannot restructure the clock tree.
    - No logic restructuring beyond the allowed transformations.
    - The output netlist must remain functionally equivalent.
    - Setup and hold timing are checked across the specified MCMM scenarios.
    - Logic-cell displacement from the initial legal placement is limited.

- The workflow before and after integrating the contestant's tool is shown in Figure 1.
    - ISPD27 Contest Workflow Steps
        - **Initial Flow Execution**: OpenROAD completes global placement, resizing and buffering, detailed placement, and clock tree synthesis (CTS). The post-CTS design is legalized before being provided to contestants.
        - **Export to Contestants**: The framework exports the following files to contestants' tools:
            - .v file: Post-CTS gate-level netlist with fixed clock tree.
            - .def file: legalized post-CTS placement.
            - .sdc files: timing constraints for the specified operating modes.
            - .lib files: timing libraries for the specified PVT corners.
            - .lef files: physical definitions of cells and routing layers.
            - MCMM configuration: definitions of the mode-corner scenarios used for timing analysis.
        - **Contestant Optimization**: Contestants' tools perform repeater optimization, gate sizing / VT selection, and equivalent input-pin swapping. The provided clock tree must remain unchanged. Contestants must output the modified .v and .def.
        - **Evaluation Flow**: The submitted design first undergoes legality, logical-equivalence, clock-tree-integrity, and transformation checks.
            - **If the design passes validation:**
              Global routing → parasitic extraction → MCMM OpenSTA timing and power analysis → QoR evaluation.
            - **If the design fails validation:**
              The submission is treated as an evaluation failure according to the contest rules.

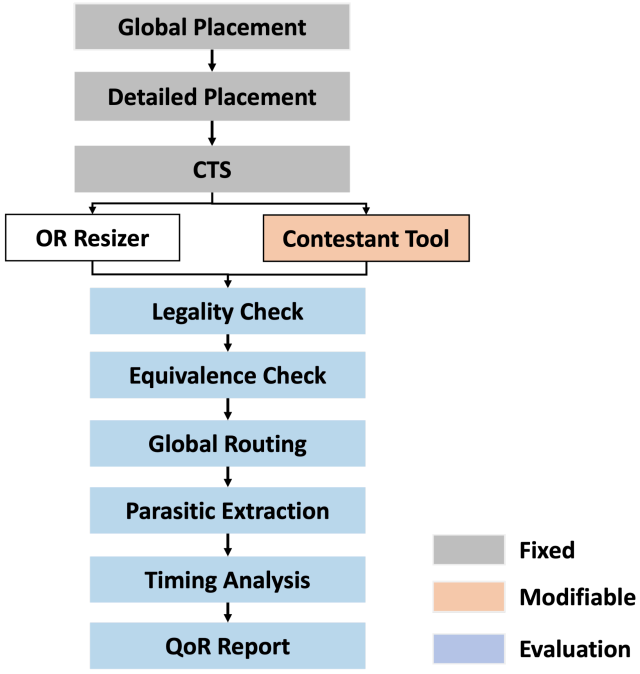


Figure 1. ISPD 2027 Contest flow. The contestant's buffering and sizing tool replaces the OpenROAD Resizer step. Fixed placement and CTS stages (gray) provide inputs; the evaluation flow (blue) performs equivalence checking, global routing, parasitic extraction and timing analysis to measure QoR.

3. Problem Formulation

3.1 Input Formats

Each benchmark will include standard design files in the **ASAP7** technology node. The provided files include:

- **.lef file**: Specifies the library definitions of the different cells, metal layers, and placement layout of the design
- **.lib file**: Timing and power libraries for multiple PVT corners.
- **.sdc file**: Timing constraints.
- **.v file:** Post-CTS gate-level netlist including the fixed clock tree.
- **.def file**: Legalized post-CTS placement.
- **MCMM configuration (.tcl)**: Associates each timing scenario with its SDC, Liberty libraries, and interconnect RC settings

The contest evaluates one physical design under **multiple timing scenarios**. Each scenario has its own PVT libraries, SDC constraints, and interconnect RC settings. Every testcase has three functional scenarios, one per corner. Scan testcases include three additional scan-test scenarios at the same corners, with scan enable = 1.

Scenario	Corner	Process	Voltage	Temperature	SDC
Functional-BC	FF_0P77V_0C	FF	0.77 V	0°C	BC.sdc
Functional-TC	TT_0P70V_25C	TT	0.70 V	25°C	TC.sdc
Functional-WC	SS_0P63V_100C	SS	0.63 V	100°C	WC.sdc
Scan-test-BC (scan testcases only)	FF_0P77V_0C	FF	0.77 V	0°C	BC_test.sdc
Scan-test-TC (scan testcases only)	TT_0P70V_25C	TT	0.70 V	25°C	TC_test.sdc
Scan-test-WC (scan testcases only)	SS_0P63V_100C	SS	0.63 V	100°C	WC_test.sdc

- Each SDC specifies the scenario's clock periods and setup/hold uncertainties. These requirements may differ across scenarios.
- Benchmarks are calibrated so that multiple scenarios impose meaningful setup and hold constraints.
- Both setup and hold are evaluated in every scenario. Physical clocks are propagated through the supplied clock tree. No additional OCV derating is applied.
- Contestants must preserve the official SDCs, libraries, RC settings, and scenario definitions.
- **Scan testcases:** *In the functional scenarios*, scan-shift paths ending at the scan input (SI) of a scan flop are set as false paths. *In the scan-test scenarios*, the scan clock uses the clock period defined for the corresponding PVT corner; the functional data input (D) of every register and all hard-macro inputs are set as false paths, and scan-shift paths and I/O paths are timed.

3.2 Output Formats

Your tool must return a legalized placement.

- **Modified Verilog Netlist (.v)**: Includes allowed buffering, sizing, VT-selection, and pin-swapping changes.
- **Updated DEF File (.def)**: Reflects your placement solution

All modifications must preserve logical equivalence and obey the contest constraints.

3.3 Evaluation: Metrics, Scoring, Platforms

3.3.1 Metrics and Scoring**

The scoring metric is subject to change based on contest needs and the quality of submitted solutions. The weights of individual score components will be announced after the Alpha submission. Setup TNS improvement will remain the primary scoring objective.

Contestants will be notified before changes are made to the scoring metrics.

Contestants' solutions are evaluated based on post-global routing design quality, runtime efficiency, and legality, using OpenROAD's timing and analysis infrastructure.

The **baseline** is the supplied legalized post-CTS design without contestant or Resizer optimization. The baseline and contestant solutions are evaluated using the same evaluation flow. All design-quality metrics are normalized against the corresponding metrics of the baseline. Runtime is normalized against the OpenROAD Resizer reference flow.

Metrics are evaluated across all MCMM scenarios. Let s represent a timing scenario. The scenario weights α_s sum to one. Different scenario weights may be used for setup and hold. Functional-only testcases have three scenarios, while scan testcases have six scenarios.

Evaluation metrics include:

- Timing and power improvement
  - Normalized setup TNS improvement
    - $\Delta T_s = \frac{TNS_s - TNS_{baseline,s}}{|TNS_{baseline,s} + \epsilon|}$
    - The ϵ values are small positive constants used for numerical stability
  - Normalized setup WNS improvement
    - $\Delta WNS_s = \frac{WNS_s - WNS_{baseline,s}}{|WNS_{baseline,s} + \epsilon|}$
    - The ϵ values are small positive constants used for numerical stability
  - Normalized dynamic power and leakage power improvement
    - $\Delta P_{dyn,s} = \frac{P_{dyn,baseline,s} - P_{dyn,s}}{P_{dyn,baseline,s}}$
    - $\Delta P_{leak,s} = \frac{P_{leak,baseline,s} - P_{leak,s}}{P_{leak,baseline,s}}$
    - Note: Power is scored in the functional scenarios only. BC typically has higher dynamic power due to its higher clock frequency and supply voltage.
  - Combine with weight w_T, w_{wns}, w_{dyn} and w_{leak}
    - $S_{PPA} = \sum_s \alpha_s (w_T * \Delta T_s + w_{wns} * \Delta WNS_s + w_{dyn} * \Delta P_{dyn,s} + w_{leak} * \Delta P_{leak,s})$

- Hold penalty
  - Hold TNS penalty
    - $P_{HTNS} = \sum_s \alpha_s \frac{max(0, H_s^{TNS} - \tau_s^{TNS})}{\tau_s^{TNS} + \epsilon}$
  - Hold WNS penalty
    - $P_{HWNS} = \sum_s \alpha_s \frac{max(0, H_s^{WNS} - \tau_s^{WNS})}{\tau_s^{WNS} + \epsilon}$
  - Combine with weight w_{HTNS} and w_{HWNS}
    - $P_{hold} = w_{HTNS} P_{HTNS} + w_{HWNS} P_{HWNS}$
- ERC violations
  - For violation type $k \in \{slew,\ cap,\ fanout\}$, let $V_{k,s}$ be the sum of violation magnitudes:
    - $P_{ERC} = \sum_s \alpha_s \sum_k w_k \frac{V_{k,s} - V_{k,baseline,s}}{V_{k,baseline,s} + \epsilon}$
- Runtime
  - The runtime of the buffering & sizing tool itself
    - $R_{tool} = \frac{t_{tool} - t_{baselineTool}}{t_{baselineTool}}$
      - Where t_{tool} is the runtime of contestants' developed tools, and $t_{baselineTool}$ is the runtime of OR Resizer
  - Total flow runtime (buffering & sizing tool + fixed evaluation flow)
    - $R_{flow} = \frac{t_{flow} - t_{baselineFlow}}{t_{baselineFlow}}$
      - Where t_{flow} is the contestants' total flow runtime, and $t_{baselineFlow}$ is the default (Figure 1, left) total flow runtime
  - Combine with weight $w_{toolRuntime}$ and $w_{flowRuntime}$
    - $R = w_{toolRuntime} R_{tool} + w_{flowRuntime} R_{flow}$
  - Note: The tool runtime must not exceed **7 hours**. The total flow runtime limits will be finalized after the Alpha submission.

- Average displacement penalty
  - Average Manhattan movement of surviving original movable cells

    $P_{dis} = w_{dis} \frac{D_{avg} - D_{baselineAvg}}{D_{baselineAvg} + \epsilon}$
    - The baseline is the average displacement produced by OR Resizer
- Global routing overflow penalty
  - **Maximum** O_{max} global routing overflow values after routing (thresholds $O_{max\,Thr}$)
    - $P_{max} = \frac{max(0, O_{max} - O_{maxThr})}{max(1, O_{maxThr} + \epsilon)}$

    - **Total** O_{total} global routing overflow values after routing (thresholds $O_{total\,Thr}$)
        - $P_{total} = \frac{max(0, O_{total} - O_{totalThr})}{max(1, O_{totalThr} + \epsilon)}$
    - Combine with weight $w_{maxOverflow}$ and $w_{totalOverflow}$
        - $P_{overflow} = w_{maxOverflow} P_{max} + w_{totalOverflow} P_{total}$
- Benchmark score
    - $S_{benchmark} = S_{PPA} - P_{hold} - P_{ERC} - R - P_{dis} - P_{overflow}$

The exact weight of each term will be determined after benchmark calibration.

3.3.2 Evaluation conditions

- Clock timing is analyzed using **propagated clocks** through the supplied clock tree. After CTS, wire parasitics are estimated using a Steiner tree for each clock net, based on placed pin locations and platform RC parameters. OpenSTA combines these wire parasitics with Liberty cell models to calculate clock arrival times and skew. For official evaluation, global-routing-based parasitics replace the placement-based estimates.
- We exclude teams that submit the unmodified input DEF, directly use OpenROAD Resizer, or reuse cached solutions without making any modifications. Only teams that implement their own optimization methods are included in the ranking.
- Placement legality, netlist equivalence, clock-tree integrity, and compliance with the permitted optimization transforms will be checked. Only submissions passing all mandatory validation checks will be considered legal.
- For teams with illegal designs (i.e., failing equivalence or legality checks), the corresponding metric values are set to **1% worse than the worst value** among all teams and baseline for the same design.
    - Formally, Value_illegal = 1.01 * worst (Value_{baseline}, Value_{team})

3.3.3 Platforms

- We will provide an evaluation environment for you via the Purdue Anvil supercomputer, with a specified number of CPU and GPU hours in your allocation, after successful Alpha submission. We expect participants to otherwise use their own resources for main development, and to use this provided resource for tuning and practice evaluation.
    - Purdue Anvil provides 128-core/128-thread AMD EPYC-7763 CPUs with NVIDIA A100 GPUs with CUDA 12.6 and Apptainer.
- The final evaluation environment will have stricter limits as defined below and will be run using the Apptainer environment:
    - 1 NVIDIA A100 GPU (40 GB)
    - 16 physical CPU cores/threads
    - 64 GB memory
    - 200 GB disk
- Availability

    - Contestants **with university email addresses** can access these resources.
    - Teams that submit a working alpha can obtain resources, and those that submit a working beta can obtain more resources.
    - **Caveats:** (1) The planned resource availability is subject to change (e.g., with changes to U.S. government policy), and (2) the amount of resource per team may depend on the number of teams registering.
    - **We have provided a User Guide on how to use this resource for contestants with successful Alpha submissions.**
- The runtime limits will be finalized after the alpha submission.

We will provide a Docker script that generates the Docker image, as well as a script to convert the Docker image to an Apptainer image. Along with that, we provide Docker and Apptainer images. We will use this Apptainer image for evaluation. **Contestants must build on top of this Docker or Apptainer environment.** For more details see the submission guidelines.

4. Submission Guidelines

Environment setup. All participants must build on the Docker or Apptainer environment we provide. If additional packages are needed, include a **setup.sh** script that completes in under two hours to set up the environment. During the contest period, we will update the Docker/Apptainer environment with any major missing packages so participants do not need to install them.

Submission files. Each participant must submit a **ZIP file** containing setup.sh, run.sh and any additional data or code not already retrievable in setup.sh.

The ZIP file should be named **TeamID_solution.zip** and submitted through the official submission link. Contestants need to share the **cksum value** of the submission ZIP file via email (ispd2027contest@gmail.com).

The run.sh must invoke your developed tool to read the provided post-CTS benchmark and produce modified .def and modified .v (see output formats) in the directory specified by <output_dir>. The execution format should be:

run.sh <input_dir> <platform_dir> <output_dir> <top_module>

- *<input_dir>* contains the benchmark input files including the post-CTS .def and .v, timing constraints (.sdc), and MCMM configuration. For a scan testcase, <input_dir> is the *scan/* directory of the benchmark.
- *<platform_dir>* contains the ASAP7 enablement, in the same layout as Platform/ASAP7.
- *<output_dir>* is the empty directory in which you will place your <output_dir>/<design_name>.def and <output_dir>/<design_name>.v.
- *<top_module>* is the top module of the design.

The submitted tool must preserve the supplied clock tree and use the official timing constraints and MCMM scenario definitions.

5. Benchmarks

We will provide a diverse set of **6 public benchmarks*** **and 8 hidden benchmarks** in the ASAP7 technology node with 7.5T cell library. The benchmarks will vary in size from 15K - 1500K instances.
The eight hidden benchmarks will not be released to contestants until the contest concludes. **Final contest scores and rankings will be determined solely based on performance on the hidden benchmarks.** The hidden benchmarks are comparable in scale to the public benchmarks.

**AES and JPEG were released on October 5. AES scan, JPEG scan, Coral NPU and NyuziProcessor were released on Nov. 7. The reference results with/without OpenROAD Resizer optimization are available in postgrt_results.csv in the contest repository.*

(Name, Util)	Mode	Size	Macros (Y,N)	Clock Period (BC, TC, WC (ps))	Clock Uncertainty (Setup, hold (ps))	Wire cap scale (BC, TC, WC)
(AES, 0.5)	Functional	15,343	N	248, 308, 429	(0, 55)	1,1,1
(JPEG, 0.65)	Functional	56,061	N	407, 550, 770	(0, 35)	1,1,1
(AES, 0.5), scan	Functional and scan	15,362	N	248, 308, 429	(0, 55)	1,1,1
(JPEG, 0.65), scan	Functional and scan	56,199	N	407, 550, 770	(0, 35)	1,1,1
(Coral NPU, 0.7)	Functional	1,300,007	Y	1358,1850,2779	(0,50)	1,1,1
(NyuziProcessor, 0.55)	Functional	337,874	Y	787, 1012, 1444	(0,50)	0.9,1,1.1

- **In AES scan, Coral NPU and NyuziProcessor, hold is not checked on paths that start at an input port or end at a hard-macro input.** Setup is checked on all paths. These exclusions are written in the SDC files (set_false_path -hold).
- Some testcases scale the wire capacitance of each corner (for example ×0.9 at BC and x1.1 at WC). The scaling is defined in the *rc.tcl* file of the testcase.

6. Timeline

- Registration Open: Oct. 5, 2026
- Registration Close: Dec. 5, 2026
- Alpha Submission Deadline: Jan. 11, 2027
- Beta Submission Deadline: Feb. 15, 2027
- Final Submission Deadline: Mar. 20, 2027 (Anywhere on earth, and it is a hard deadline)

- Results Announcement: March 31, 2027

7. Contact

Email: ispd2027contest@gmail.com

8. Registration

- Please fill in this online registration form
- Registration window: Oct.5, 2026 - Dec. 5, 2026

9. Contest Prizes

- USD 1,500 / 750 / 375 and NVIDIA GPUs for the top three teams. All amounts are pre-tax and subject to applicable ACM and local tax withholding.

10. Organizers

- Junyeong Jang, Jakang Lee, Yiting Liu and Davit Markarian from UCSD (advisor: Prof. Andrew B. Kahng)

11. Sponsors

- Purdue University and the NSF Chipshub: compute resources for teams and submission evaluation
- NVIDIA: prizes for winning teams

References

[1] A. B. Kahng, S. Kang, S. Kundu, Y. Liu, D. Markarian, S. Park and Z. Wang, "Invited: Post-Placement Buffering and Sizing Contest", Proc. ISPD, 2026.
[2] OpenROAD-Flow-Scripts. https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts
[3] I. Sagar, K. B. Prakash, and G. R. Kanagachidambaresan, “PyTorch.” *Programming with TensorFlow: solution for edge computing applications* (2021), pp. 87-104.
[4] A. B. Kahng, Y. Liu, and Z. Wang, “Recursive Learning-Based Virtual Buffering for Analytical Global Placement”, *MLCAD*, 2025.
[5] Y.-C. Lu, Z. Guo, K. Kunal, R. Liang, and H. Ren, “INSTA: An Ultra-Fast, Differentiable, Statistical Static Timing Analysis Engine for Industrial Physical Design Applications”, Proc. ICCAD, 2025.
[6] Y. Du, Z. Guo, Y. Lin, R. Wang and R. Huang, “Fusion of Global Placement and Gate Sizing with Differentiable Optimization”, *Proc. ICCAD*, 2024.
[7] Y.-C. Lu, K. Kunal, G. Pradipta, R. Liang, R. Gandikota, and H. Ren, “LEGO-Size: LLM-Enhanced GPU-Optimized Signoff-Accurate Differentiable VLSI Gate Sizing in Advanced Nodes,” ISPD, 2025.

[8] Y. Ye, P. Xu, L. Ren, T. Chen, H. Yan, B. Yu, and L. Shi, “Learning-Driven Physically Aware Large-Scale Circuit Gate Sizing,” IEEE TCAD, vol. 44, no. 5, pp. 1901–1914, 2025.
[9] Y. Du, Z. Guo, R. Wang, and Y. Lin, “Differentiable Physical Optimization,” ICCAD, 2025.
[10] B-Y. Wu, R. Liang, G. Pradipta, A. Agnesina, H. Ren and V. A. Chhabria, “Invited Paper: 2024 ICCAD CAD Contest Problem C: Scalable Logic Gate Sizing using ML Techniques and GPU Acceleration”, Proc. ICCAD, 2024.
[11] H.-H. Hsiao, Y.-C. Lu, S. K. Lim, and H. Ren, “BUFFALO: PPA-Configurable, LLM-Based Buffer Tree Generation via Group Relative Policy Optimization,” ICCAD, 2025.